

TRAVAUX PRATIQUES

DevOps S8

Infrastructure AWS avec Terraform

Provisionner, versionner et automatiser une infrastructure cloud complète

Provider : **AWS + LocalStack (tests locaux)**

Phase 1 : LocalStack (local) → Phase 2 : AWS eu-west-1

Individuel ou binôme | Rendu : dépôt Git

Table des matières

1. Présentation du TP

1.1 Contexte et objectifs

Ce TP fait suite au cours Ansible. Vous allez maintenant ajouter la couche infrastructure : plutôt que de supposer qu'un serveur existe et qu'Ansible s'en occupe, vous allez le créer vous-même dans AWS avec Terraform.

1.2 Ce que vous allez construire

L'objectif final est une infrastructure cloud fonctionnelle comprenant :

- Un VPC avec subnets public et privé dans AWS
- Une instance EC2 (serveur web) avec Security Group configuré
- Une paire de clés SSH gérée par Terraform
- Un bucket S3 (stockage fichiers statiques ou remote state)
- En bonus : une base de données RDS et un Load Balancer

⚠ Attention

L'ensemble de votre code Terraform sera versionné dans un dépôt Git. Terraform destroy sera lancé à la fin de chaque séance pour ne pas consommer les crédits AWS inutilement.

1.3 Résumé des livrables

	Objectif
✓	Dépôt Git avec structure Terraform propre et README explicatif
✓	Provider AWS configuré - LocalStack (tflocal) puis AWS réel
✓	VPC avec subnet public, Internet Gateway et Route Table
✓	Security Group : SSH (22), HTTP (80), HTTPS (443)
✓	Instance EC2 t3.micro avec clé SSH et IP publique
✓	Bucket S3 avec versioning activé
✓	Outputs : IP publique EC2, ID VPC, nom bucket
✓	Variables et fichier terraform.tfvars (sans secrets)
✓	terraform plan propre (0 error) + terraform apply réussi
✓	Connexion SSH à l'instance EC2 démontrée
★	Remote state sur S3 + locking DynamoDB
★	Module réutilisable pour la partie réseau (VPC + subnets)
★	Base de données RDS (db.t3.micro) en subnet privé
★	Application Load Balancer devant l'EC2
★	User data : script de bootstrap de l'instance au démarrage
★	Chaînage Terraform → Ansible : provisioning de l'EC2 via playbook

	Objectif
★	Multi-environnements : workspace dev et prod avec variables distinctes
★	tflint sans erreurs

2. Exercices - Providers gratuits

Avant de déployer sur AWS, vous allez pratiquer les fondamentaux de Terraform avec deux providers entièrement gratuits : Docker (local, aucun compte) et GitHub (compte gratuit existant). La logique HCL, les commandes et le workflow sont identiques - seul le provider change.

2.1 Provider Docker - Infra locale

Docker est parfait pour apprendre Terraform sans aucun prérequis cloud. Il tourne sur votre machine avec Docker Desktop.

Prérequis : Docker Desktop installé et démarré.

Initialiser le projet

```
mkdir tp-docker && cd tp-docker
touch versions.tf main.tf variables.tf outputs.tf terraform.tfvars
```

versions.tf

```
terraform {
  required_version = ">= 1.6"
  required_providers {
    docker = {
      source  = "kreuzwerker/docker"
      version = "~> 3.0"
    }
  }
}

provider "docker" {} # Connexion au daemon Docker local - aucun argument
```

main.tf - Déployer nginx dans Docker

```
resource "docker_image" "nginx" {
  name = "nginx:alpine"
}

resource "docker_network" "app" {
  name = "app-network"
}

resource "docker_container" "web" {
  name     = "${var.project_name}-web"
  image    = docker_image.nginx.image_id

  ports {
    internal = 80
    external = var.host_port
  }

  networks_advanced {
    name = docker_network.app.name
  }
}
```

```
}
}
```

variables.tf

```
variable "project_name" {
  description = "Nom du projet"
  type        = string
  default     = "tp-terraform"
}

variable "host_port" {
  description = "Port sur la machine hôte"
  type        = number
  default     = 8080
}
```

outputs.tf

```
output "container_name" {
  value = docker_container.web.name
}

output "url" {
  value = "http://localhost:${docker_container.web.ports[0].external}"
}
```

Lancer et tester

```
terraform init
terraform plan
terraform apply

# Vérifier que nginx répond
curl http://localhost:8080
```



Conseil

Essayez de changer la valeur de `host_port` dans `terraform.tfvars` (ex: 9090) puis relancez `terraform apply`. Terraform détruira et recréera le container. C'est ce qu'on appelle un "forces remplacement" - vérifiez-le dans le plan !

Exercices à réaliser

- Ajouter un 2e container (ex: redis:alpine) sur le même réseau Docker
- Passer le nom de l'image en variable et la modifier sans tout détruire
- Vérifier l'idempotence : lancer `terraform apply` deux fois, 0 changes au 2e passage
- Lancer `terraform destroy` et vérifier que le container disparaît de Docker Desktop

2.2 Provider GitHub - GitOps

Le provider GitHub permet de gérer des dépôts, branches protégées, teams et secrets GitHub Actions. Vous avez déjà un compte GitHub - c'est suffisant.

Créer un Personal Access Token

GitHub → Settings → Developer settings → Personal access tokens → Fine-grained tokens → Generate new token

Permissions minimales requises :

- Repository permissions : Contents (Read & Write), Administration (Read & Write), Metadata (Read), Secrets (Read & Write)
- Durée : 30 jours (suffisant pour le TP)

```
# Stocker le token en variable d'environnement (ne jamais le mettre dans le code !)
export GITHUB_TOKEN="ghp_xxxxxxxxxxxxx"
```

versions.tf

```
terraform {
  required_version = ">= 1.6"
  required_providers {
    github = {
      source  = "integrations/github"
      version = "~> 6.0"
    }
  }
}

provider "github" {
  token = var.github_token # ou lit GITHUB_TOKEN automatiquement
}
```

main.tf - Créer et protéger un dépôt

```
resource "github_repository" "app" {
  name          = "${var.project_name}-demo"
  description   = "Dépôt géré par Terraform - DevOps 4A"
  visibility    = "public"
  has_issues    = true
  auto_init     = true # Initialise avec un README
}

resource "github_branch_protection" "main" {
  repository_id = github_repository.app.node_id
  pattern       = "main"

  required_pull_request_reviews {
    required_approving_review_count = 1
    dismiss_stale_reviews           = true
  }
}
```

```
resource "github_actions_secret" "db_url" {
  repository      = github_repository.app.name
  secret_name     = "DATABASE_URL"
  plaintext_value = var.db_url
}
```

variables.tf

```
variable "github_token" {
  description = "Personal Access Token GitHub"
  type        = string
  sensitive   = true
}

variable "project_name" {
  description = "Nom du projet"
  type        = string
}

variable "db_url" {
  description = "URL de base de données (pour le secret GitHub Actions)"
  type        = string
  sensitive   = true
  default     = "postgresql://user:pass@localhost/mydb"
}
```

Appliquer et vérifier

```
terraform init
terraform plan
terraform apply

# Le dépôt apparaît dans github.com/<votre-compte>/
# Vérifier : Settings → Branches → Branch protection rules
# Vérifier : Settings → Secrets and variables → Actions
```

Exercice bonus - terraform import

Importez un nouveau dépôts que vous aurez créé manuellement sur Github dans le state Terraform :

```
# 1. Déclarer la ressource dans main.tf (avec le bon nom)
resource "github_repository" "existing" {
  name = "mon-repo-existant"
}

# 2. Importer l'état réel dans Terraform
terraform import github_repository.existing mon-repo-existant

# 3. Vérifier que le plan est propre (0 changes)
terraform plan
```

Note

terraform import est la commande qui permet de mettre sous contrôle Terraform une ressource déjà existante sans la recréer. C'est un cas très courant en entreprise quand on veut "terraformer" une infrastructure existante.

Exercices à réaliser

- Créer 2 dépôts avec des paramètres différents (avec/sans wiki ou autre)
- Ajouter un topic au dépôt : topics = ["terraform", "devops"]
- Supprimer la protection de branche avec terraform apply et observer le changement dans GitHub
- (Bonus) Importer un repo existant avec terraform import

Attention

terraform destroy supprimera les dépôts GitHub ! Faites attention de ne pas détruire un vrai projet. Utilisez des noms de dépôts clairement identifiés comme TP (ex: tp-terraform-demo).

3. Mise en place et infrastructure de base

3.1 Mise en place de l'environnement

Cette partie se déroule en deux phases distinctes :

- Phase 1 - LocalStack : vous écrivez et testez tout votre code Terraform localement, sans compte AWS, sans coût, sans risque.
- Phase 2 - AWS réel : une fois le code validé sur LocalStack, vous le déployez sur AWS avec vos crédits (\$100 offerts à l'inscription).

Conseil

Faites toujours valider votre code sur LocalStack avant de l'appliquer sur AWS réel. Cela évite de consommer des crédits sur du code qui ne fonctionne pas encore.

Phase 1 - Installer et démarrer LocalStack

LocalStack est un émulateur AWS qui tourne dans Docker sur votre machine. Il simule les services AWS (EC2, S3, VPC, RDS...) sans aucun compte.

Prérequis : Docker doit être installé et démarré sur votre machine.

Installer LocalStack ainsi que tflocal (terraform-local)

```
# Démarrer LocalStack (en arrière-plan)
localstack start -d
```

```
# Vérifier que les services sont prêts
localstack status services
curl http://localhost:4566/_localstack/health
```

Note

LocalStack Community Edition (gratuite) supporte : EC2, S3, VPC, IAM, SQS, DynamoDB, Route53, ACM, et bien d'autres. Pour RDS, l'édition Pro est requise - utilisez directement AWS réel pour la BDD.

Phase 1 - Utiliser tflocal à la place de terraform

tflocal est un wrapper de la commande terraform qui redirige automatiquement toutes les requêtes vers LocalStack (<http://localhost:4566>) sans modifier votre code HCL.

```
# Remplacer terraform par tflocal pour la phase LocalStack
tflocal init
tflocal plan
tflocal apply
tflocal destroy
```

Pour LocalStack, Terraform a besoin de fausses credentials (n'importe quelle valeur) :

```
export AWS_ACCESS_KEY_ID=test
export AWS_SECRET_ACCESS_KEY=test
export AWS_DEFAULT_REGION=us-east-1
```

⚠ Attention

Certaines ressources ne se comportent pas exactement comme sur AWS réel dans LocalStack. Notamment : les instances EC2 démarrent mais ne sont pas vraiment accessibles par SSH. Cela est suffisant pour valider la configuration Terraform, pas pour tester l'application.

Phase 2 - Créer votre compte AWS et configurer les credentials

Une fois votre code validé sur LocalStack, vous allez le déployer sur AWS réel.

1. Créez un compte AWS sur aws.amazon.com (carte bancaire requise pour vérification, mais \$100 de crédits offerts à l'activation)
2. Connectez-vous à la console AWS → Informations d'identification de sécurité → Créer une clé d'accès
3. Configurez AWS CLI avec vos clés :

```
aws configure
AWS Access Key ID      : AKIA...
AWS Secret Access Key  : ...
Default region name    : eu-west-3
Default output format  : json
```

Vérifiez que la connexion fonctionne :

```
aws sts get-caller-identity
```

Phase 2 - Passer de tflocal à terraform

Aucune modification de votre code HCL n'est nécessaire - remplacez simplement tflocal par terraform dans vos commandes :

```
# Phase 2 : commandes standard Terraform
terraform init
terraform plan
terraform apply
```

⊘ Important

Les crédits AWS (\$100) sont partagés sur toute la durée du cours. Lancez impérativement terraform destroy à la fin de chaque séance. Une instance EC2 t3.micro oubliée coûte ~\$0.01/h soit ~\$7/mois - multipliez par toute la classe.

3.2 Structure du projet Terraform

Créez un dépôt Git vide (GitLab ou GitHub) et clonez-le localement. Initialisez la structure suivante :

```
mkdir tp-terraform-aws && cd tp-terraform-aws
git init # ou cloner votre dépôt distant
```

Créez les fichiers de base :

```
touch versions.tf provider.tf main.tf variables.tf outputs.tf
touch terraform.tfvars .gitignore README.md
```

Contenu du .gitignore - à ne jamais oublier :

```
.terraform/
*.tfstate
*.tfstate.backup
*.tfplan
.terraform.lock.hcl  # optionnel, certains équipes le commitent
*.auto.tfvars        # si contient des secrets
```

⚠ Important

Ne jamais commiter de fichiers contenant des credentials AWS (access key, secret key, session token). Ces informations ne doivent jamais apparaître dans l'historique Git.

3.3 Configurer le provider AWS

Remplissez versions.tf avec la configuration Terraform et le provider requis :

```
# versions.tf
terraform {
  required_version = ">= 1.6"
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}
```

Puis le fichier provider.tf :

```
# provider.tf
provider "aws" {
  region = var.aws_region

  default_tags {
    tags = {
      Project      = var.project_name
      Environment  = var.environment
      ManagedBy    = "Terraform"
      Owner        = var.owner
    }
  }
}
```

Le bloc default_tags est une bonne pratique AWS : toutes vos ressources hériteront automatiquement de ces tags, sans avoir à les répéter sur chaque ressource.

Initialiser le projet :

```
terraform init
```

i Note

Terraform va télécharger le provider hashicorp/aws (~100 Mo). Vérifiez qu'il n'y a pas d'erreur. Le dossier .terraform/ est créé - il est dans le .gitignore.

3.4 Variables et valeurs par défaut

Définissez vos variables dans variables.tf :

```
# variables.tf
variable "aws_region" {
  description = "Région AWS cible"
  type        = string
  default     = "eu-west-3"
}

variable "project_name" {
  description = "Nom du projet (utilisé dans les tags et noms de ressources)"
  type        = string
}

variable "environment" {
  description = "Environnement (dev, staging, prod)"
  type        = string
  default     = "dev"
  validation {
    condition     = contains(["dev", "staging", "prod"], var.environment)
    error_message = "L'environnement doit être dev, staging ou prod."
  }
}

variable "owner" {
  description = "Nom ou email du responsable"
  type        = string
}

variable "vpc_cidr" {
  description = "CIDR du VPC"
  type        = string
  default     = "10.0.0.0/16"
}

variable "public_subnet_cidr" {
  description = "CIDR du subnet public"
  type        = string
  default     = "10.0.1.0/24"
}

variable "instance_type" {
  description = "Type d'instance EC2"
  type        = string
  default     = "t3.micro"
}

variable "my_ip" {
  description = "Votre IP publique pour autoriser SSH (format CIDR : x.x.x.x/32)"
  type        = string
}
```

Créez terraform.tfvars avec vos valeurs (ce fichier est chargé automatiquement) :

```
# terraform.tfvars
project_name = "tp-terraform-votreprenom"
owner        = "prenom.nom@efrei.fr"
environment  = "dev"
my_ip        = "X.X.X.X/32"  # Remplacez par votre IP publique
```

Pour trouver votre IP publique :

```
curl ifconfig.me
```

3.5 Créer le réseau - VPC et Subnet

Ajoutez dans main.tf le réseau de base. Nous allons créer un VPC avec un subnet public.

```
# main.tf

# — Data sources —
data "aws_availability_zones" "available" {
  state = "available"
}

# — VPC —
resource "aws_vpc" "main" {
  cidr_block           = var.vpc_cidr
  enable_dns_hostnames = true
  enable_dns_support   = true

  tags = { Name = "${var.project_name}-vpc" }
}

# — Internet Gateway —
resource "aws_internet_gateway" "main" {
  vpc_id = aws_vpc.main.id
  tags   = { Name = "${var.project_name}-igw" }
}

# — Subnet public —
resource "aws_subnet" "public" {
  vpc_id                = aws_vpc.main.id
  cidr_block             = var.public_subnet_cidr
  availability_zone       = data.aws_availability_zones.available.names[0]
  map_public_ip_on_launch = true

  tags = { Name = "${var.project_name}-public-subnet" }
}

# — Route Table (accès internet depuis subnet public)
resource "aws_route_table" "public" {
  vpc_id = aws_vpc.main.id

  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.main.id
  }
}
```

```

}

tags = { Name = "${var.project_name}-public-rt" }
}

resource "aws_route_table_association" "public" {
  subnet_id      = aws_subnet.public.id
  route_table_id = aws_route_table.public.id
}

```

Vérifier avant d'appliquer

```

terraform fmt      # Formater le code
terraform validate  # Vérifier la syntaxe
terraform plan      # Voir ce qui va être créé

```

Lisez attentivement le plan. Vous devez voir 5 ressources à créer (+). Si tout est correct :

```
terraform apply
```



Conseil

Tapez 'yes' quand Terraform vous demande confirmation. Après l'apply, vérifiez dans la console AWS (VPC → Your VPCs) que votre VPC est bien créé.

3.6 Security Group - Le pare-feu de l'instance

Le Security Group définit les règles de trafic entrant (ingress) et sortant (egress) de vos instances EC2. Ajoutez dans main.tf :

```

resource "aws_security_group" "web" {
  name          = "${var.project_name}-web-sg"
  description   = "Security Group pour le serveur web"
  vpc_id        = aws_vpc.main.id

  # SSH - uniquement depuis votre IP
  ingress {
    from_port = 22
    to_port   = 22
    protocol  = "tcp"
    cidr_blocks = [var.my_ip]
    description = "SSH depuis ma machine"
  }

  # HTTP public
  ingress {
    from_port = 80
    to_port   = 80
    protocol  = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
    description = "HTTP public"
  }

  # HTTPS public
  ingress {

```

```

    from_port = 443
    to_port   = 443
    protocol  = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
    description = "HTTPS public"
  }

  # Tout le trafic sortant autorisé
  egress {
    from_port = 0
    to_port   = 0
    protocol  = "-1"
    cidr_blocks = ["0.0.0.0/0"]
    description = "Tout le trafic sortant"
  }

  tags = { Name = "${var.project_name}-web-sg" }
}

```

Conseil

Restreindre SSH à votre seule IP (var.my_ip) est une bonne pratique de sécurité. En entreprise, on restreint encore davantage via un bastion host ou un VPN.

3.7 Clé SSH et instance EC2

Pour vous connecter à l'EC2, Terraform doit importer votre clé SSH publique dans AWS.

Générer une paire de clés SSH (si vous n'en avez pas)

```

ssh-keygen -t ed25519 -C "tp-terraform" -f ~/.ssh/tp_terraform
# Crée ~/.ssh/tp_terraform (clé privée) et ~/.ssh/tp_terraform.pub (clé publique)

```

Importer la clé publique dans AWS via Terraform

```

resource "aws_key_pair" "deployer" {
  key_name     = "${var.project_name}-key"
  public_key   = file("~/ssh/tp_terraform.pub")
}

```

Trouver l'AMI Ubuntu 24.04 via un data source

```

data "aws_ami" "ubuntu" {
  most_recent = true
  owners      = ["099720109477"] # Canonical

  filter {
    name   = "image-id"
    values = ["ami-0adb8ca49015e0901"]
  }
}

```

Créer l'instance EC2

```

resource "aws_instance" "web" {

```

```

ami                = data.aws_ami.ubuntu.id
instance_type      = var.instance_type
subnet_id          = aws_subnet.public.id
vpc_security_group_ids = [aws_security_group.web.id]
key_name           = aws_key_pair.deployer.key_name

root_block_device {
  volume_size = 20
  volume_type = "gp3"
  encrypted   = true
}

tags = { Name = "${var.project_name}-web" }
}

```

```

terraform plan
terraform apply

```

3.8 Outputs - Exporter les informations utiles

Les outputs permettent d'afficher et d'utiliser les valeurs importantes après l'apply. Remplissez outputs.tf :

```

# outputs.tf
output "vpc_id" {
  description = "ID du VPC créé"
  value       = aws_vpc.main.id
}

output "instance_public_ip" {
  description = "IP publique de l'instance EC2"
  value       = aws_instance.web.public_ip
}

output "instance_public_dns" {
  description = "DNS public de l'instance EC2"
  value       = aws_instance.web.public_dns
}

output "ssh_command" {
  description = "Commande SSH pour se connecter"
  value       = "ssh -i ~/.ssh/tp_terraform ubuntu@${aws_instance.web.public_ip}"
}

```

Affichez les outputs après l'apply :

```

terraform output
terraform output instance_public_ip # Valeur seule

```

3.9 Bucket S3

Ajoutez un bucket S3 pour stocker des fichiers statiques ou servir de remote state :

```

resource "aws_s3_bucket" "assets" {

```

```

bucket = "${var.project_name}-assets-${random_id.suffix.hex}"

tags = { Name = "${var.project_name}-assets" }
}

resource "random_id" "suffix" {
  byte_length = 4
}

resource "aws_s3_bucket_versioning" "assets" {
  bucket = aws_s3_bucket.assets.id
  versioning_configuration { status = "Enabled" }
}

resource "aws_s3_bucket_server_side_encryption_configuration" "assets" {
  bucket = aws_s3_bucket.assets.id
  rule {
    apply_server_side_encryption_by_default {
      sse_algorithm = "AES256"
    }
  }
}

```

Note

Le provider random doit être ajouté dans required_providers (source : hashicorp/random). Relancez terraform init après l'ajout.

3.10 Connexion SSH à l'instance

Une fois l'apply terminé, connectez-vous à votre instance :

```

# Récupérer l'IP
terraform output instance_public_ip

# Se connecter (la commande est dans l'output ssh_command)
ssh -i ~/.ssh/tp_terraform ubuntu@<IP_PUBLIQUE>

```

Sur l'instance, vérifiez :

```

# Sur l'instance EC2
uname -a          # Version du kernel Ubuntu
curl ifconfig.me  # IP publique vue de l'intérieur
df -h             # Volume de 20 Go disponible

```

3.11 Fin de séance - Destruction des ressources

À la fin de la séance, détruisez TOUTES les ressources pour ne pas consommer les crédits :

```
terraform destroy
```

Important

Tapez 'yes' pour confirmer. Sur AWS réel : vérifiez dans la console que toutes les ressources sont supprimées (EC2, VPC, S3...). Vos crédits (\$100) sont partagés - chaque ressource oubliée les grignote.

4 Travaux bonus

Bonus A - Base de données RDS

Ajoutez un subnet privé et une instance RDS PostgreSQL. La BDD ne doit PAS avoir d'IP publique et ne doit être accessible que depuis le Security Group de l'EC2.

Ressources à créer :

- `aws_subnet` (privé, `map_public_ip_on_launch = false`)
- `aws_db_subnet_group` (requis par RDS)
- `aws_security_group` pour la BDD (port 5432 depuis le SG EC2 uniquement)
- `aws_db_instance` (db.t3.micro, postgres, `skip_final_snapshot = true`)

Ajoutez un output `db_endpoint` et passez-le à Ansible via le template d'inventaire.

Bonus B - Application Load Balancer

Ajoutez un ALB devant l'EC2. L'ALB sera le point d'entrée public, l'EC2 n'aura plus besoin d'être directement accessible sur le port 80.

Ressources à créer :

- Deuxième subnet public dans une autre AZ (obligatoire pour l'ALB)
- `aws_lb` (type = application)
- `aws_lb_target_group` (port 80, protocol HTTP)
- `aws_lb_listener` (port 80 → forward vers le target group)
- `aws_lb_target_group_attachment` (associe l'EC2 au target group)

Bonus C - Terraform Workspaces (Multi-environnements)

Les workspaces permettent de gérer plusieurs environnements avec le même code. Créez un workspace dev et un workspace prod avec des tailles d'instances différentes.

```
terraform workspace new dev
terraform workspace new prod
terraform workspace select dev
```

Adaptez les variables selon le workspace dans `variables.tf` :

```
locals {
  instance_type = {
    dev = "t3.micro"
    prod = "t3.small"
  }
}

# Utiliser : local.instance_type[terraform.workspace]
```

Bonus D - tfLint

tfLint est un linter Terraform qui détecte les erreurs de configuration AWS (types d'instances invalides, AMIs dépréciées, règles de sécurité...).

```
# Installation
```

```
curl -s
https://raw.githubusercontent.com/terraform-linters/tflint/master/install_linux.sh |
bash

# Initialiser avec le plugin AWS
tflint --init

# Linter le projet
tflint --recursive
```

Note

Créez un fichier `.tflint.hcl` à la racine pour configurer les plugins. Le plugin AWS analyse les types d'instances, les AMIs, et les règles spécifiques à AWS.

Fin de séance - Nettoyage obligatoire

Avant de partir, détruisez toute l'infrastructure. Chaque ressource oubliée consomme des crédits.

```
terraform workspace select dev # Si workspaces utilisés
terraform destroy -auto-approve
```

Vérifiez dans la console AWS que les ressources suivantes ont bien été supprimées :

- EC2 instances → 0 instance running
- VPC → seul le VPC par défaut AWS doit rester
- RDS → 0 instance (si créée)
- S3 → votre bucket assets (le state bucket peut rester)
- DynamoDB → votre table de locks

5. Rendu et évaluation

5.1 Modalités de rendu

- Travail individuel ou en binôme (noms dans le README)
- Rendu : URL du dépôt Git (public ou accès donné à l'enseignant)
- Le dépôt doit contenir l'ensemble du code Terraform, propre et structuré

5.2 Contenu attendu du README.md

4. Titre, description et noms des membres
5. Prérequis (Terraform, AWS CLI, versions)
6. Instructions pas à pas pour déployer l'infrastructure
7. Explication de la structure des fichiers
8. Capture d'écran ou log de terraform apply réussi
9. Capture d'écran de la connexion SSH à l'EC2
10. Liste des bonus implémentés
11. Instructions de destruction (terraform destroy)

5.4 Conseils pour réussir

- Commitez après chaque étape fonctionnelle - l'historique Git compte
- Lisez TOUJOURS le plan avant d'appliquer. Un 'forces replacement' = recréation de la ressource
- Nommez toutes vos ressources avec des tags Name cohérents
- Un terraform destroy propre en fin de séance vaut autant qu'un terraform apply réussi
- Si quelque chose ne marche pas, aws ec2 describe-instances et les logs CloudWatch sont vos amis
- terraform state list et terraform state show <resource> sont très utiles pour déboguer

Annexe A - Commandes de référence rapide

Workflow quotidien

```
# Démarrage de session
export AWS_ACCESS_KEY_ID="..."
export AWS_SECRET_ACCESS_KEY="..."
# Pas de SESSION_TOKEN avec un IAM user (seulement avec STS/rôles)

terraform init      # 1 fois par projet / après ajout provider
terraform fmt       # Avant chaque commit
terraform validate  # Avant chaque apply
terraform plan      # OBLIGATOIRE avant apply
terraform apply     # Créer / mettre à jour
terraform output    # Voir les outputs
terraform destroy   # FIN DE SÉANCE - OBLIGATOIRE
```

Debug et inspection

```
terraform state list          # Lister les ressources
terraform state show aws_instance.web # Détail d'une ressource
terraform output -json        # Outputs en JSON
terraform output -raw instance_public_ip
TF_LOG=DEBUG terraform plan  # Mode verbeux
terraform graph | dot -Tpng > graph.png # Graphe de dépendances
```

AWS CLI utile

```
aws sts get-caller-identity # Vérifier les credentials
aws ec2 describe-instances --query 'Reservations[].Instances[].{ID:InstanceId,IP:PublicIpAddress,State:State.Name}'
aws s3 ls                  # Lister les buckets
aws ec2 describe-vpcs     # Lister les VPCs
aws logs tail /aws/ec2/user-data # Logs user data
```

Annexe B - Structure complète du projet

```
tp-terraform-aws/
├── .gitignore
├── README.md
├── backend.tf          # Remote state (après séance 2)
├── versions.tf         # Version Terraform + providers requis
├── provider.tf         # Configuration du provider AWS
├── main.tf             # Ressources principales
├── variables.tf        # Déclaration de toutes les variables
├── outputs.tf          # Valeurs exportées
├── terraform.tfvars    # Valeurs (sans secrets !)
├── modules/
│   ├── network/       # Module réseau réutilisable
│   └── main.tf
└──
```

```
├── variables.tf
├── outputs.tf
├── scripts/
│   └── user_data.sh    # Script de bootstrap EC2
├── templates/
│   └── inventory.tpl    # Template inventaire Ansible
```

Annexe C - Checklist de rendu

☐	Le dépôt Git est accessible (public ou accès donné à l'enseignant)
☐	Le README contient les noms, les instructions et les captures d'écran
☐	Aucune credential AWS dans le code ou l'historique Git
☐	Le .gitignore exclut .terraform/, *.tfstate, *.tfplan
☐	terraform fmt ne remonte aucune différence (code formaté)
☐	terraform validate passe sans erreur
☐	Un terraform plan + apply propre est documenté (capture ou log)
☐	La connexion SSH à l'EC2 est documentée
☐	terraform destroy a bien été lancé en fin de séance
☐	Toutes les ressources AWS ont des tags Name cohérents
☐	Les bonus implémentés sont listés dans le README

Bonne chance - et n'oubliez pas le terraform destroy !